

XVIII Data Academy

Git and GitHub Fundamentals

Agenda

1. Version Control Systems (VCS)
2. Git concepts
3. Git commands and file states
4. GitHub
5. Git workflows
6. Best practices
7. Resources

Version Control Systems (VCS)

Centralized VCS and Distributed VCS

Very Confusing System

Name	Date modified
 My Document Really Final.txt	02/03/2026 18:03
 My Document Final Final.txt	02/03/2026 18:00
 My Document Final.txt	02/03/2026 17:59
 My Document v2.txt	02/03/2026 17:59
 My Document v1.txt	02/03/2026 17:58

Version Control System (VCS)

- A tool that tracks and manages changes to files over time
- Every change is recorded
 - **What** changed
 - **Who** changed it
 - **When** in changed
 - **Why** it changed
- A change can be reverted
- Multiple people can work on features separately and combine them later without overwriting each other
- Used by software developers, designers, writers, DevOps teams, ...



Merged PR 6585: #15469 Replace ge_lib with 3 new modules
#15469 Replace ge_lib with 3 new modules
Merged PR 6547: Update setup instructions
Delete files that were merged into setup_dataquality
Remove reference to hive_metastore
Format setup_dataquality and switch the order of 2 lines
Add notebook with merged setup instructions, edit comments in config file
 origin/item_15345 Update python_formatter.yml for Azure Pipelines
move to right folder
fix trigger and pr configuration
yaml pipeline to run black and comment PR
Merged PR 6537: Setup to build wheel
Rename modules to bi4all_dq_modules
Update description and keywords in setup.py
Edit import modules
Edit init file
Add setup and build_dq2adk_wheel
Add comment with alternative import of wheel
Update import ge_lib, add init for package initialization
Migrate content not related with wheel from dqadk4all and DQ2ADK repos
Added README.md

16 Apr 2024 14:07	Miguel Félix	48b08cce
16 Apr 2024 10:38	mfelix@bi4all.pt	7a19138d
4 Apr 2024 16:52	Miguel Félix	87fef334
4 Apr 2024 15:18	mfelix@bi4all.pt	92634542
4 Apr 2024 15:09	mfelix@bi4all.pt	1b37ad6f
4 Apr 2024 15:01	mfelix@bi4all.pt	7dc7ecf1
4 Apr 2024 14:56	mfelix@bi4all.pt	a796c7ad
3 Apr 2024 15:59	Tiago Lima	096984f4
3 Apr 2024 15:30	Tiago Lima	8b844d88
3 Apr 2024 15:21	Tiago Lima	beaadafd
3 Apr 2024 15:18	Tiago Lima	f8c547d3
3 Apr 2024 15:05	Miguel Félix	35c8a95c
3 Apr 2024 14:58	mfelix@bi4all.pt	59fa0279
3 Apr 2024 14:42	mfelix@bi4all.pt	176e12a0
2 Apr 2024 18:31	mfelix@bi4all.pt	f8334d1c
2 Apr 2024 17:52	mfelix@bi4all.pt	1b83782c
2 Apr 2024 17:04	Miguel Félix	5fba18bf
1 Apr 2024 11:42	mfelix@bi4all.pt	9c2aa4f3
28 Mar 2024 18:38	mfelix@bi4all.pt	1037908e
28 Mar 2024 15:21	Miguel Félix	58dd6ae0
28 Mar 2024 11:24	Miguel Félix	0685511c

Types of Version Control Systems

- **Centralized VCS (CVCS)**

- One central server stores the project's history [single point of failure]
- The client has only the current snapshot
- If offline, there is no project's history beyond cached info [limitation]
- Example: Apache Subversion (SVN)

- **Distributed VCS (DVCS)**

- The project's history is stored both on the client and on the server [resilient solution]
- Works offline [sync with the server when online]
- Flexible and most popular approach
- Example: Git

Git concepts

Tools, setup and terminology

Git

- **Git** is a free and open-source distributed version control system (DVCS)
- Released in 2005, it is used by 96% of professional developers ([2022 Stack Overflow developer survey](#))
- Built to be efficient and fast



Git tools

- Although the core Git project is just a command-line tool, many **GUIs, editor integrations,** and **command line tools** have been built to make working with Git more convenient
- Some GUI clients (free; Windows and Mac):
 - GitHub Desktop
 - SourceTree
- Some IDEs with built-in Git integration (free; Windows, Mac and Linux):
 - Visual Studio Code
 - PyCharm

Git setup – installation

- Download and install the latest version of Git on your machine:
<https://git-scm.com/install/>
- To verify the installation, open Command Prompt (cmd) and run:
`git --version`
- You should see something like this:
`git version x.xx.x`

Git setup – configuration

- Set your username and email address in Git:
`git config --global user.name "Your Name"`
`git config --global user.email "your@email.com"`
- Confirm that you have set the Git username and email address correctly:
`git config --global user.name`
`git config --global user.email`

Git terminology (1)

- **Repository (repo):** The directory where Git keeps all the history and metadata for a project, which needs to be created once:
 - Either locally (then push to a Git hosting platform)
 - Or by cloning an existing repository
- **Remote:** A common repository on a Git hosting platform that people use to exchange their changes, called `origin` by default
- **Clone:** A local version of a repository
- **Working tree:** The set of nested directories and files in the repository that contain the project that's being worked on

Git terminology (2)

- **Hash:** A number produced by a hash function that represents the contents of a file as a fixed number of digits
- **Commit:** (Subject) A specific version of the working tree; (verb) to make a commit
- **Tag:** a name that can optionally be attached to a commit
- **Branch:** A named series of linked commits that typically form a feature, which can later be merged into the default branch
 - The most recent commit on a branch is called the **head**
 - The default branch, which is created when you initialize a repository, is usually called **main**
 - The head of the current branch is named **HEAD**

Git terminology (3)

- **Pull request:** a place to compare and discuss the differences introduced on a branch with reviews, comments, integrated tests, and more

Git commands and file states

Basic commands, main file states and
lifecycle of file states

Git commands

- Git operations are performed by using commands like:

```
git status -s
```

```
git status --short
```

- In these examples:

- `git` is the **command**

- `status` is the **subcommand**, which specifies the operation you want Git to perform

- `-s` and `--short` are both **options**, which use hyphens (-) or double hyphens (--)

Basic Git commands – repositories

- `git init`

Turns an existing directory into a Git repository

Adds a hidden subfolder within the existing directory that houses the internal data structure required for version control

- `git clone [url]`

Downloads a repository that already exists, including all the files, history and branches

Basic Git commands – branches

- `git branch [branch-name]`
Creates a new branch
- `git checkout [branch-name]`
Switches to the specified branch and updates the working directory
- `git merge [branch]`
Combines the specified branch's history into the current branch
It is usually done in pull requests
- `git branch -d [branch-name]`
Deletes the specified branch

Basic Git commands – make changes

- `git add [file]`
Snapshots/stages the file in preparation for versioning

Any changes that are staged will become a part of the next snapshot and a part of the project history

- `git commit -m "[descriptive message]"`
Saves the snapshot permanently to the project history

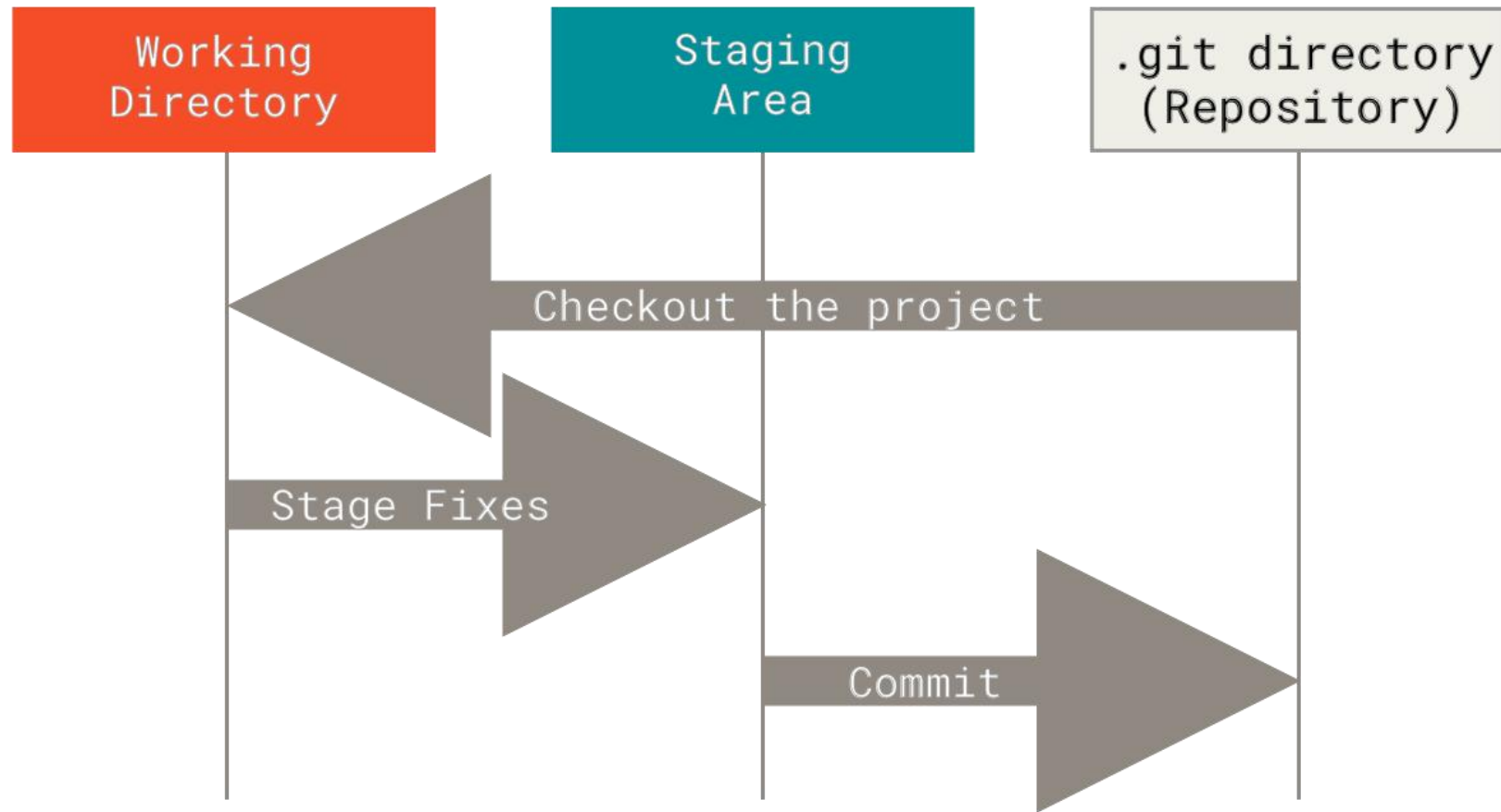
Staging and committing separately gives developers complete control over the history of their project

- `git status`
Shows the status of changes as untracked, modified or staged

Main file states (1)

- Git has three main states that your files can exist in:
 - **Modified** – You have changed a file but have not told Git about it yet. The file exists in your project folder (**working directory**).
 - **Staged** – You have marked a modified file in its current version to go into your next commit snapshot. The file exists in the **staging area** / **index**. Only staged changes will go into the next commit.
 - **Committed** – The file is safely stored in Git's history. The file snapshot is saved permanently in the local **repo**.
- It is possible to skip the staging area using the -a option, but this may cause you to include unwanted changes:
`git commit -a -m "[descriptive message]"`

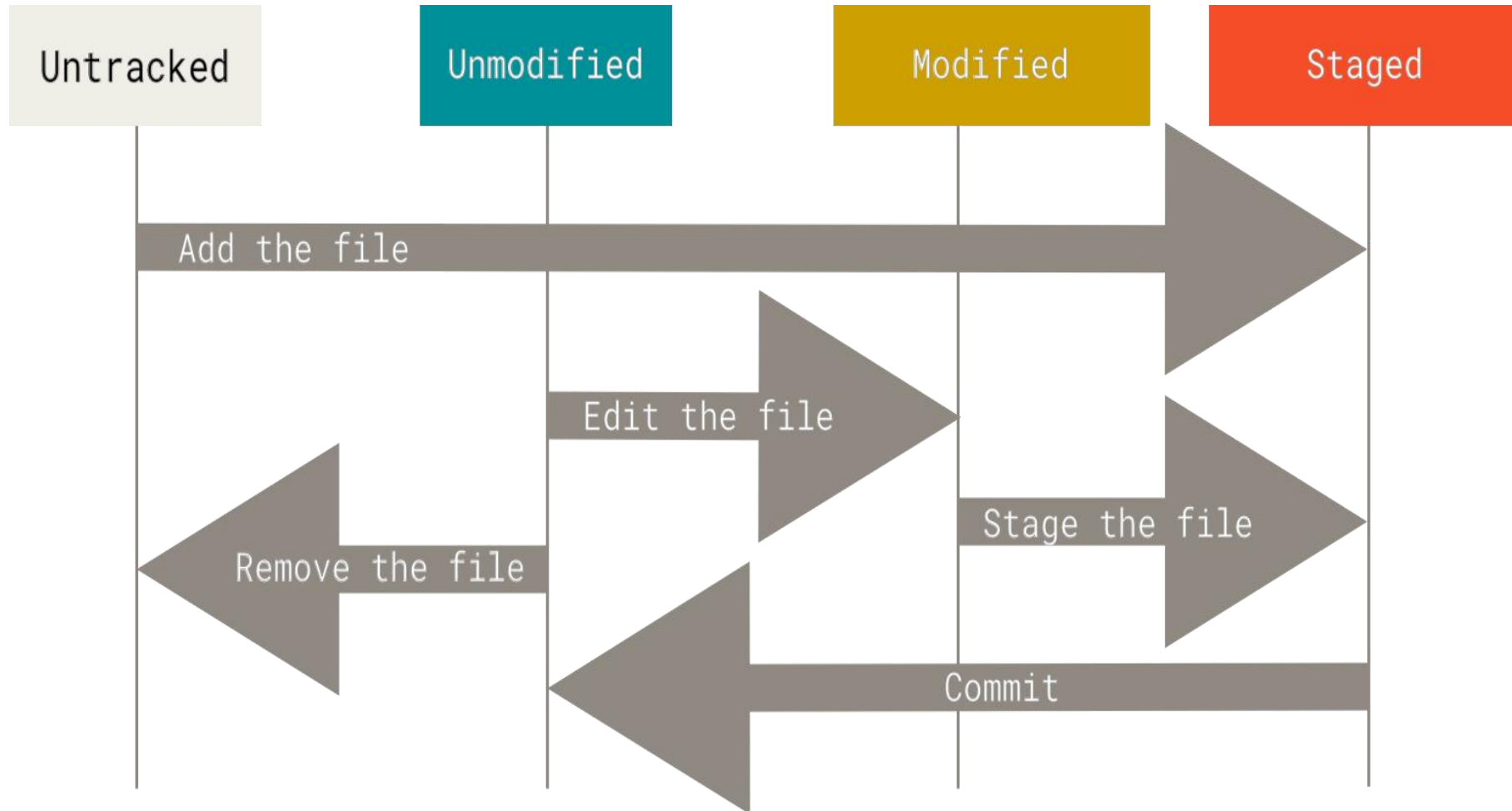
Main file states (2)



Lifecycle of file states (1)

- **Tracked** files are files that were in the last snapshot, as well as any newly staged files. They can be:
 - Unmodified
 - Modified
 - Staged
- **Untracked** files are everything else: any files in your working directory that were not in your last snapshot and are not in your staging area

Lifecycle of file states (2)

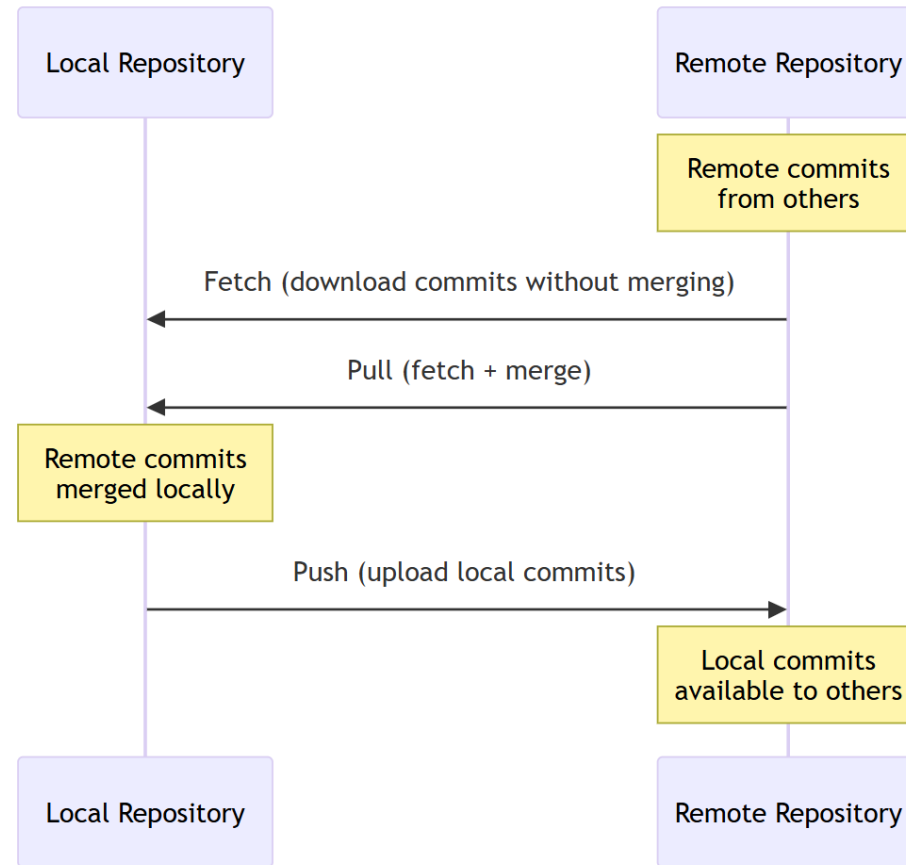


Basic Git commands – synchronize changes (1)

- `git fetch`
Downloads all history from the remote tracking branches
- `git merge`
Combines remote tracking branch into current local branch
- `git pull`
Updates the current local working branch with all new commits from the corresponding remote branch

It is a combination of `git fetch` and `git merge`
- `git push`
Uploads all local branch commits to the remote repository

Basic Git commands – synchronize changes (2)



GitHub

Git hosting, features and interface

Git hosting

- There are multiples services that allow users to store and manage their Git repositories online
- Some popular options:
 - GitHub
 - Azure Repos
 - GitLab
 - Bitbucket

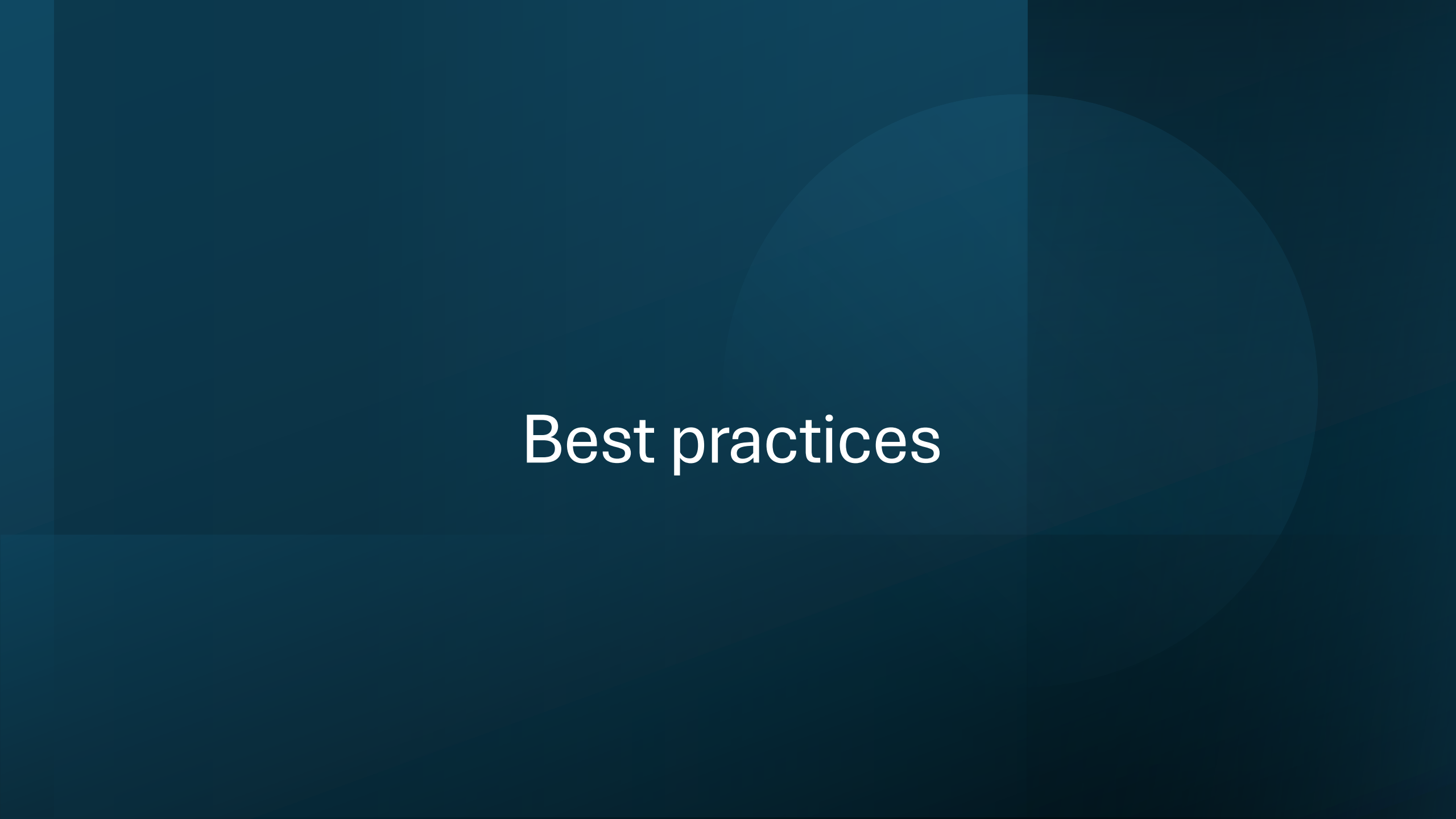


GitLab

ATLASSIAN
Bitbucket

GitHub

- **GitHub** is a proprietary cloud-based platform that uses Git at its core
- Allows users to create, store, manage and share their code
- Provides access control, bug tracking, feature requests, task management, continuous integration, and wikis for every project
- Popular choice to host open-source projects
- Key features:
 - Code
 - Issues
 - Pull requests
 - Discussions
 - Actions
 - Projects
 - Wiki
 - Notifications



Best practices

Best practices

- **Follow the selected Git workflow and branch naming conventions** – For instance: [branch-type]/[Task-ID]-[task_title]
- **Pull before push** – Before starting work, ensure your local branch is up to date and reduce the risk of conflicting with changes from others.
- **Write good commit messages** – Give each commit a descriptive message to help you and future contributors understand what changes the commit contains. Use imperative mode. Mention the task ID when possible.
- **Use atomic commits** – Ideally, each commit contains an isolated and complete change. This makes it easy to revert your changes if you decide to take a different approach.
- **Don't commit secrets** – Avoid the inclusion of sensitive information, such as API keys or passwords; use a vault instead.
- **Use .gitignore** – Don't store unnecessary files in the repo; keep it clean and performant.
- **Delete old branches** – After merging a pull request, delete the branch to prevent the team from accidentally using it in the future.

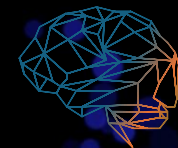
Resources

Resources (1)

- Git: <https://git-scm.com/>
 - Learn – Git Cheat Sheet: <https://git-scm.com/cheat-sheet>
 - Tools: <https://git-scm.com/tools>
- GitHub Training Kit: <https://training.github.com/>
 - Git Cheat Sheet (multiple languages)
 - GitHub Training Manual
- GitHub Foundations course (Microsoft Learn):
<https://learn.microsoft.com/en-us/training/courses/gh-900t00>
- GitHub Docs – Get started: <https://docs.github.com/en/get-started>

Resources (2)

- Visual Studio Code: <https://code.visualstudio.com/>
 - Documentation (Setup, Get Started, Source Control):
<https://code.visualstudio.com/docs>
 - Git Graph extension:
<https://marketplace.visualstudio.com/items?itemName=mhutchie.git-graph>



LISBOA

Av. Marechal Gomes da Costa
Nº27 A/B - Armazém 1
1800-255 Lisboa

+351 217 266 165

PORTO

TECMAIA
Rua Eng. Frederico Ulrich, 2650
Edifício Genesis
Piso 0 - Fração A019
4470-605 Maia

+351 217 266 165

hello@bi4all.pt

